

Talk-to-Data Delivery Blueprint — Master

Daniel Brule · [linkedin.com/in/danielbrule](https://www.linkedin.com/in/danielbrule)
v1.0 · June 2026

- 1 Executive summary
- 2 How to read this blueprint
 - 2.1 Intended audience
 - 2.2 Master, phase documents and annexes
 - 2.3 Companion implementation repository
 - 2.4 Scope and specialist review
- 3 Talk-to-Data concept
 - 3.1 What it is / what it is not
 - 3.2 Where T2D creates value
 - 3.3 Where T2D is the wrong tool
 - 3.4 Target user flow
 - 3.5 Text-to-SQL vs RAG-over-analytics
 - 3.6 Bounded agentic AI
 - 3.7 Key risks and safe failure
 - 3.7.1 Multi-turn conversation handling
- 4 Delivery blueprint overview
 - 4.1 Delivery depth and proportionality
 - 4.2 Delivery principles
 - 4.3 Canonical control definitions
 - 4.4 Phase overview and decision gates
 - 4.5 Delivery flow
 - 4.6 Cross-phase outputs
 - 4.7 Illustrative semantic artefact: approved metric card
- 5 Project governance and operating model
 - 5.1 Minimum governance model
 - 5.2 Decision gates
 - 5.2.1 Decision outcomes
 - 5.2.2 Illustrative evaluation expectations
 - 5.3 Budget, capacity and run cost
 - 5.3.1 Development and operational involvement
 - 5.3.2 Indicative cost per query
 - 5.3.3 Key budget risks
 - 5.4 Typical team shape
 - 5.5 Build, buy or partner considerations
 - 5.6 Operating model after launch
 - 5.7 Adoption and trust
- 6 How to adapt the blueprint to a real project
 - 6.1 Start with project intent
 - 6.2 Calibrate delivery depth
 - 6.3 Use decision gates, not fixed timelines
 - 6.4 Decide what to simplify, strengthen or defer
 - 6.5 Practical adaptation checklist
- 7 Final practitioner view

1 Executive summary

Enterprise Talk-to-Data (T2D) is not a chatbot connected to a database. It is a governed decision interface over trusted data.

The value is clear: business users can ask follow-up questions, explore performance, and get faster answers without waiting for a dashboard change or joining an analyst queue. Used well, T2D can reduce friction between business questions and trusted answers, especially where users need flexible exploration rather than fixed reporting.

The risk is equally clear. If definitions, data sources, permissions and answer quality are not controlled, the system can return confident, plausible and wrong answers. In an enterprise context, that is not a minor limitation; it is a trust, governance and adoption failure.

This blueprint sets out how to move from an idea to a capability that can be trusted, operated and improved in a real

organisation and is built around five delivery principles:

- **Use T2D where conversation adds value.** T2D is strongest where users need flexible exploration, frequent follow-up questions and governed answers that existing reporting channels cannot provide quickly or easily.
- **Focus on the real risk areas.** T2D delivery risk concentrates around framing, semantics, evaluation and governance. Common failure points include unclear use cases, inconsistent definitions, weak evidence of answer quality, unsafe access and no clear ownership after launch.
- **Deliver proportionately.** A proof of concept can be lightweight, but any move towards pilot or production requires stronger evidence, clearer ownership and more formal controls.
- **Reassess early and regularly.** Pausing, narrowing, redirecting or stopping the initiative is a valid outcome if the use case lacks value, the data is not ready, semantics cannot be agreed, security cannot be controlled, or the organisation is not ready to operate the capability.
- **Operationalise the system around the model.** Monitoring, access control, cost control, support, regression testing and continuous improvement matter as much as the LLM itself.

The purpose of this blueprint is therefore not to promote T2D for every analytical need. It is to help teams decide where T2D is appropriate, what must be true for it to succeed, and how to deliver it safely and pragmatically.

Key decisions to be made:

- Is T2D the right solution, or would a dashboard, report or analyst workflow be better?
- Is the use case valuable enough to justify the data, semantic, security and operating effort?
- Is there enough evidence to move from POC to MVP, pilot or production?
- Who will own the capability, cost, risk and improvement cycle after launch?

The Master Guide provides the strategic delivery logic; the phase documents explain how to execute each stage, and the annexes provide optional working tools.

2 How to read this blueprint

This Master Guide explains the delivery logic for building a governed Talk-to-Data capability. It is not intended to be read as a rigid methodology or a complete implementation manual. Teams should adapt the blueprint to their business context, data maturity, governance model, technology stack, security requirements and delivery constraints.

2.1 Intended audience

This document is intended for the teams responsible for shaping, approving, delivering and operating a T2D capability: business sponsors, product owners, delivery leads, data and analytics teams, architects, security and governance stakeholders, and adoption leads.

2.2 Master, phase documents and annexes

Use the Master Guide to understand the overall delivery logic: where T2D creates value, where it is inappropriate, what risks must be controlled and how the delivery phases fit together.

Use the phase documents for practical delivery guidance. Use the annexes selectively for templates, checklists, matrices and examples. The annexes are optional working material; they are not intended to be read end to end.

2.3 Companion implementation repository

A companion implementation repository is maintained separately. This blueprint defines the delivery method, governance logic and evidence model. The implementation repository demonstrates selected patterns through a reference Talk-to-Data build.

2.4 Scope and specialist review

This document provides a delivery blueprint for T2D initiatives. It is not a technical design, legal assessment, security policy, compliance review, vendor selection framework or full project plan.

Security, privacy, regulatory, contractual and audit requirements should be reviewed by the appropriate specialists for each

organisation.

3 Talk-to-Data concept

T2D is the ability for users to ask questions about governed business data in natural language and receive reliable, explainable answers without writing SQL, building a report or navigating a dashboard.

At its simplest, a user may ask: “What was revenue last month by region?” A production T2D system must then interpret the question, identify the approved metric and dimensions, apply the user’s access rights, query the trusted data layer, validate the result, and return an answer with appropriate context and caveats. The user can then ask follow-up questions within the approved scope.

The important point is that T2D is not just a chatbot connected to a database. It is a governed analytics capability with a conversational interface. Reliable answers depend on approved definitions, trusted data sources, access controls, query validation, answer evaluation, monitoring and safe failure handling.

For example, if a commercial user asks for revenue, the system must know which revenue definition is approved, which period and region are in scope, and what data the user is authorised to see. If a finance user asks for stock, the system must understand whether the question refers to physical stock, available stock, reserved stock or stock value.

The objective is not only to make data easier to access. It is to make business answers faster, safer and more consistent.

3.1 What it is / what it is not

Talk-to-Data is	Talk-to-Data is not
A governed way to ask questions of trusted business data	A generic chatbot connected to all enterprise data
A conversational analytics capability	A replacement for all dashboards and reports
A way to accelerate exploration and decision support	A substitute for analyst judgement or data governance
A controlled interface using approved definitions and access rules	A free-form AI tool with unrestricted access to data
A capability that improves over time through feedback and semantic refinement	A static tool that works out of the box without ongoing maintenance
A system that should fail safely when it cannot answer	A system that should guess when the answer is uncertain

3.2 Where T2D creates value

T2D creates value when users need flexible, trusted exploration rather than fixed reporting. It is most useful where business questions change frequently, users need follow-up analysis, and existing dashboards or analyst workflows are too slow for the decision being made.

Strong candidate areas include executive performance review, sales and commercial analysis, finance exploration, customer analytics, operations performance and root-cause investigation.

T2D is usually appropriate when three conditions are present:

- **The use case has clear decision value.** The capability supports real business decisions, not generic curiosity.
- **The data and semantics are mature enough.** Relevant sources, metrics, definitions, joins and access rules can be made explicit.
- **Conversation adds value.** Users need quick flexible exploration, clarification and follow-up questions that are not already well served by existing BI or analyst workflow.

3.3 Where T2D is the wrong tool

T2D should not be used simply because GenAI is available. It is often the wrong tool for stable, repeatable and highly standardised reporting, where dashboards or scheduled reports are cheaper, clearer and easier to govern.

It is also a poor fit where the organisation cannot agree metric definitions, cannot identify trusted data sources, cannot enforce access controls, or cannot support the capability after launch.

For complex judgement-based analysis, T2D may support exploration, but it should not replace analyst ownership. The system can help retrieve, summarise and test data-driven hypotheses, but human judgement remains necessary where interpretation, context and accountability matter.

3.4 Target user flow

- **Understand the question.** Classify the user intent, business domain, metric, filters and expected output.
- **Clarify where needed.** Ask for missing information when the question is ambiguous, incomplete or unsafe.
- **Retrieve governed context.** Use approved metadata, metric definitions, dimensions, joins, caveats and access rules.
- **Generate and validate the query.** Produce a query only within approved scope, then check syntax, joins, filters, permissions, row limits and cost.
- **Execute against trusted data.** Run the query against the governed queryable layer, not uncontrolled source systems.
- **Validate and explain the result.** Check whether the result is plausible, empty, unexpected or restricted, then return the answer with source, assumptions and caveats.
- **Log and improve.** Capture questions, queries, errors, feedback and evaluation outcomes for monitoring and continuous improvement.

3.5 Text-to-SQL vs RAG-over-analytics

Most T2D systems combine two patterns:

Pattern	What it does	Typical use	Main risk
Text-to-SQL	Translates a user question into a governed query against structured data.	Numeric answers, aggregations, filtering, slicing and business metrics.	Wrong joins, wrong grain, wrong filters, unsafe query generation or overconfident numeric answers.
RAG-over-analytics	Retrieves approved metadata, definitions, metric documentation, caveats or precomputed analytical content to ground the answer.	Metric interpretation, business definitions, caveats, policy explanations and contextual guidance.	Retrieved context may be incomplete, stale, irrelevant or treated as authoritative when it is not.

These patterns should not be treated as interchangeable. Text-to-SQL is usually needed for live structured analysis. Retrieval is usually needed to ground the system in approved definitions, caveats and semantic context.

A strong T2D design uses retrieval to constrain and explain the query, not to invent numeric answers from documents. Numeric answers should usually come from governed data execution, not generated prose.

3.6 Bounded agentic AI

T2D can be viewed as a bounded form of agentic AI. In a basic implementation, the system translates a natural-language question into a governed query and returns an answer. In a more advanced implementation, it may select the relevant domain, retrieve metric definitions, choose tools, ask clarification questions, generate and validate queries, inspect results, apply caveats and decide whether it is safe to answer.

This creates value because the system can manage more of the analytical flow. It also increases risk: more intermediate decisions, more tool calls, more context drift, higher cost, harder debugging and greater exposure risk.

For this reason, agentic T2D should remain bounded by approved tools, approved data sources, query limits, access rules, evaluation tests, monitoring and escalation paths. It should not be treated as an unconstrained autonomous agent.

Agentic behaviour does not remove accountability. Product, data, security, AI and operating owners remain accountable for scope, definitions, access, model behaviour, monitoring and release control.

3.7 Key risks and safe failure

Risk	Failure mode	Required control
Weak use case	T2D is built because GenAI is attractive, not because conversation improves a real workflow	Framing around decisions, users, priority questions and alternatives
Semantic ambiguity	Correct-looking answer uses the wrong business meaning for revenue, margin, stock or customer	Approved metric definitions, dimensions, caveats and ownership
Query error	The system uses the wrong grain, join, filter, time window or aggregation	Governed query surface, SQL validation, join rules and golden questions
Access or inference leakage	Users see or infer restricted information through rows, columns, filters or follow-ups	Row/column controls, masking, aggregation limits, audit logs and exposure tests
False confidence	The system gives a fluent answer when it should clarify, caveat, refuse or escalate	Safe-failure rules, answer format controls and evaluation cases
Weak evaluation	There is no evidence that answers are correct, grounded or safe	Golden questions, expected answers, unsafe cases and

		regression tests
Prompt or retrieval attack	User input or retrieved content attempts to override rules or expose restricted data	Prompt isolation, tool permissions, retrieval controls and logging
No operating owner	The capability degrades after launch as data, metadata, models and users change	Named product, data, AI, security and support ownership

For numeric and financial answers, fluency is not evidence of correctness. A T2D answer should make visible the source, metric definition, filters, time period, caveats and limits used to produce the result.

3.7.1 Multi-turn conversation handling

Multi-turn conversation is a major source of T2D value and risk. Follow-up questions can preserve useful context, but they can also create context drift, accidental scope expansion, stale filters or access leakage.

A T2D design should define what carries across turns, when context resets, when clarification is required and how the conversation remains auditable. Multi-turn support should improve exploration without allowing unsupported reasoning, uncontrolled data exposure or hidden changes in scope.

4 Delivery blueprint overview

Delivering T2D requires more than building a natural-language interface. It requires coordinated work across business framing, data readiness, semantic modelling, governed data foundations, architecture, security, evaluation, adoption and operations.

The delivery logic is simple: start with a bounded business problem, make the data and semantics explicit, build only against governed foundations, test the system against agreed questions and failure modes, then scale only when trust, security and ownership are proven.

A T2D initiative should not progress because time has passed or a prototype looks impressive. It should progress because the evidence is strong enough that value is safely created.

4.1 Delivery depth and proportionality

The phases in this blueprint describe the work required to deliver a T2D capability. They should be scaled to the project's intent, risk and intended use.

A discovery exercise or POC may use a lighter version of the phases to test whether the opportunity is valuable and feasible. An MVP, pilot or production path requires progressively stronger evidence, clearer ownership and more formal controls.

[Section 6.1](#) sets out the maturity levels used in this blueprint, from discovery and POC through to MVP, pilot, production readiness and ongoing run. The practical question for each project is therefore not only which activities are needed, but how much evidence, control and operational readiness are required before the capability is used more widely or for higher-risk decisions.

4.2 Delivery principles

Principle	Description
Start from business questions, not model capability.	T2D should be shaped around priority questions, decisions and workflows, not around the desire to deploy GenAI.
Use T2D only where conversation changes the workflow.	T2D should be used where follow-up questions, flexible exploration or governed conversational access create value beyond dashboards, reports or analyst-led workflows. See section 3.3 for cases where T2D is the wrong tool.
Do not connect the model to unmanaged data.	T2D should query governed sources, approved metrics and controlled semantic context, not raw enterprise data by default.
Make semantics explicit before scaling.	Metrics, dimensions, filters, joins, grains and caveats must be defined well enough for the system to use them consistently.
Treat evaluation as a launch gate, not a nice-to-have.	Golden questions, expected answers and failure categories are required before pilot or production.
Design for safe failure.	The system must ask, refuse, caveat or escalate when it cannot answer safely.
Control exposure, not just access.	It is not enough to authenticate users; the system must control what they can infer through queries, aggregations and follow-ups.
Keep humans accountable.	Business owners, analysts and data teams should validate definitions, outputs and exceptions, especially during early adoption.

Productionise the system around the model.	Monitoring, logging, regression testing, prompt and version control, cost management, support and ownership determine whether the service survives.
Reassess continuously.	Narrowing, pausing or stopping the initiative is valid if value, data readiness, security or operating maturity are not strong enough.

These principles are what separate a working POC from a sustainable product. A T2D prototype can often be built quickly, but turning it into a trusted, scalable and supportable capability is significantly more complex. Data quality, semantics, security, evaluation, cost control, adoption and operational support must be designed into the delivery from the start.

Stopping or pausing a project after framing should not be seen as failure. A well-run framing phase can create value even if the project does not proceed immediately. It may expose data management gaps, inconsistent metric definitions, unclear ownership, weak governance, unrealistic expectations or missing business value. An early stop can be a successful outcome if it prevents poorly founded delivery and helps the organisation assess GenAI opportunities more rigorously.

The model is often less important than the end-to-end flow around it. A stronger model may improve performance, but it will not compensate for weak framing, unclear semantics, poor data foundations, unsafe access controls, missing evaluation or poor operational ownership.

4.3 Canonical control definitions

Term	Definition
Evaluation	Testing answer quality, query correctness, grounding, access behaviour and safe failure against agreed questions and failure cases
Golden question	A representative question with expected answer, accepted caveats, required sources and known failure modes
Regression testing	Re-running agreed tests after changes to prompts, models, metadata, data, SQL rules, access rules or orchestration
Monitoring	Live tracking of usage, answer quality, errors, refusals, latency, cost, support issues and risk signals
Traceability	Ability to connect a user question to retrieved metadata, model calls, generated SQL, validation result, executed query, final answer and feedback
Access lifecycle	Joiner, mover, leaver and role-change control for T2D users and permissions
Release boundary	Approved users, data domains, question types, answer types, caveats, usage limits, support model and escalation conditions
Residual risk	Known risk accepted by accountable owners after validation and before pilot or production exposure
Controlled change	Versioned, tested and approved changes to prompts, models, metadata, tools, validation rules, access rules or answer behaviour
Safe failure	Behaviour where the system asks, caveats, refuses or escalates instead of guessing or exposing unsupported answers

4.4 Phase overview and decision gates

Phase	Indicative timeline ¹	Core decision	Main evidence before progressing
1. Frame	Days 1 - 3	Confirm the business problem, target users, priority questions, MVP boundary and success criteria.	Clear use case, decision value, scope, owner and funding route.
2. Assess data and semantics	Week 1	Understand whether the required data, definitions, joins, filters and access rules can support the use case.	Known sources, metric definitions, data gaps, ownership and semantic risks.
3. Build governed foundations	Weeks 1 - 3	Create the trusted queryable layer and approved semantic context for T2D.	Curated data assets, approved metrics, access model and query boundaries.
4. Design architecture	Weeks 2 - 3	Define the target T2D architecture, including orchestration, metadata retrieval, tool boundaries, query generation, validation, execution and response flow.	Approved architecture, tool boundaries, logging, guardrails and integration approach.
5. Prototype / MVP build	Weeks 3 - 5	Build a bounded MVP from the approved orchestration design to prove user value, AI patterns, answer quality, operating controls and production gaps.	Working MVP, validation evidence, logs, cost/quality findings and backlog.
6. Validation, assurance and remediation	Weeks 4 - 6	Validate the MVP against agreed safety, access, answer-quality, query, auditability and operational requirements; remediate blockers and approve the pilot boundary.	Evaluation results, security/access validation, failure categories, remediation and retest evidence, residual-risk decision and pilot constraints.
7. Controlled pilot and user testing	Weeks 5 - 7+	Test the validated MVP with controlled users, real workflows, trust, support and operating processes	Trace-linked feedback, adoption/trust signals, support/access issues, usage risks, backlog and Phase 8 gaps
8. Production readiness and controlled release	Weeks 7 - 14	Fix release-critical issues and confirm scope, controls, support, onboarding, cost, ownership and release conditions	Release sign-off, regression evidence, runbooks, support model and cost/risk approval.

9. Operate, adopt and improve	Weeks 14 -22+	Stabilise usage, govern changes, refresh semantics and manage safe improvement or stop.	Feedback loop, regression tests, change log, roadmap and stop route.
-------------------------------	---------------	---	--

Note: The phase model should be read as a delivery logic, not a fixed waterfall. Several phases may be performed lightly for a discovery or POC, repeated or deepened for an MVP, and formalised before pilot or production readiness. The important discipline is to avoid carrying POC assumptions into MVP, pilot or production without revalidating them

4.5 Delivery flow

The delivery flow is iterative rather than strictly linear. Framing sets the initial scope, but data readiness, semantic modelling and architecture often develop together. A prototype may expose missing definitions. Evaluation may reveal a query-grain issue. Pilot feedback may show that users ask questions differently from the framing workshop.

These loops should be expected. The discipline is not to avoid iteration; it is to make sure each iteration improves scope, semantics, controls or evidence before the capability is exposed to more users.

4.6 Cross-phase outputs

Some outputs are created, refined and reused across multiple phases of the T2D delivery. These artefacts provide continuity between phases.

Cross-phase output	Why it matters	Main phases involved
Use-case scope and MVP boundary	Defines what the first release will and will not cover. It prevents uncontrolled expansion of domains, users and questions.	Framing, pilot, production readiness, scale decisions
Priority question set	Provides mandatory list of business questions the capability must answer. Drives data, semantic modelling, evaluation and testing.	Framing, data readiness, governed data layer, evaluation, pilot
Approved metrics and semantic definitions	Ensures business terms, KPIs, filters, dimensions and calculations are interpreted consistently.	Data readiness, governed data layer, evaluation, pilot, continuous improvement
Trusted queryable layer	Provides controlled data foundation that T2D system can safely query.	Data readiness, governed data layer, architecture, prototype, production readiness
Access and exposure model	Defines which users can access which data, metrics, rows, columns and levels of detail.	Framing, data readiness, security validation, pilot, production readiness
Risk, decision and assumption log	Tracks unresolved issues, key decisions, open assumptions and dependencies across the project.	All phases
Operating model	Defines who owns the capability, who supports it, how issues are handled, and how improvements are managed after go-live.	Framing, production readiness, adoption and stabilisation, continuous improvement
Golden question and regression set	Evolves priority questions into golden questions, expected answers, caveats, failure cases and regression tests. It provides the evidence base for answer quality, safe failure and future change control.	Framing, data and semantic readiness, prototype build, security and quality validation, pilot, production readiness, continuous improvement

These artefacts are the connective tissue of the delivery. They prevent T2D from becoming a disconnected prototype and provide the evidence needed to move from experiment to production.

In a real implementation, these artefacts may live across semantic layers, data catalogues, repositories, metadata APIs or structured configuration files. The format matters less than whether the system can retrieve, apply, test and govern it consistently.

Note: Golden questions should mature across the delivery lifecycle. Framing creates candidate questions. Data and semantic readiness determine which questions are eligible for evaluation. Prototype and validation turn them into expected answers and failure cases. Production readiness converts them into regression tests.

4.7 Illustrative semantic artefact: approved metric card

A production T2D system needs more than a metric name. It needs enough semantic context for the system to interpret, query, explain and caveat the answer consistently.

Field	Example
Metric name	Net revenue
Business definition	Revenue after discounts, credits and refunds.

Calculation logic	Sum of net invoice amount.
Grain	Invoice line.
Default filters	Exclude test accounts and cancelled invoices.
Approved dimensions	Month, region, product, customer segment.
Source of truth	Finance revenue mart.
Owner	Finance.
Access rules	Restricted by region and legal entity.
Caveats	Month-end adjustments may change values until close.
Example approved question	“What was net revenue last month by region?”
Validation example	Net revenue for the last closed month reconciles to the finance pack

A metric card like this is not documentation for its own sake. It is part of the control that allows the system to select the right data, apply the right filters, explain the answer and know when to ask for clarification.

5 Project governance and operating model

Governance is not an afterthought in Talk-to-Data. It determines which questions can be answered, which definitions are trusted, which users can access which data, what evidence is required before exposure, and who owns the capability after launch.

A strong model, clean interface and working SQL generation are not enough if ownership is unclear. The governance model should answer six practical questions:

- Who owns the use case and business value?
- Who approves the data, metrics and semantic definitions?
- Who is accountable for security, privacy and access control?
- Who decides whether answer quality is good enough to progress?
- Who owns the cost model, usage controls and operating budget?
- Who owns the capability after launch?

Without clear answers, T2D can become an impressive prototype with no safe route to production. A more detailed governance and role matrix can be maintained in the annexes.

5.1 Minimum governance model

The governance model should be proportionate to the project risk. A narrow prototype may require lightweight governance, while a production capability serving sensitive data, regulated data or large user groups will require clearer decision gates and separation of responsibilities.

Governance area	Typical owner	Key accountability
Business sponsorship	Executive sponsor	Owns strategic rationale, funding and senior stakeholder alignment.
Product ownership	Product owner	Owns use-case scope, MVP boundary, priorities, user needs and delivery trade-offs.
Data ownership	Data owner	Approves source-of-truth decisions, data usage and access routes.
Metric and semantic ownership	Metric owner / domain owner	Approves KPI definitions, dimensions, filters, calculation logic and caveats.
Security and compliance	Security / governance / compliance lead	Approves access controls, exposure rules, audit requirements and regulatory constraints.
Technical ownership	AI architect / solution architect	Owns orchestration, metadata retrieval, query generation, validation and integration design.
Evaluation ownership	Product owner / QA lead / business SME	Owns golden questions, expected answers, failure categories and answer-quality evidence.
Operational ownership	Product owner / platform owner / support owner	Owns monitoring, support, cost control, issue management, launch and continuous improvement after launch.

The detailed role matrix should be adapted to the organisation and maintained separately.

5.2 Decision gates

A T2D project should not progress only because a prototype works or the timeline has elapsed. It should progress because the required evidence and approvals are in place.

Decision gate	Core question	Evidence required
Framing approval	Is the use case valuable, bounded and worth pursuing?	Business problem, target users, priority questions, MVP boundary, success criteria and funding route.
Data and semantic readiness	Are the data and definitions good enough to build safely?	Source mapping, metric definitions, joins, filters, data gaps, ownership and access rules.
Architecture approval	Is the technical design controlled and observable?	Orchestration design, tool boundaries, metadata retrieval, query validation, logging and guardrails.
Security approval	Can the system control access and exposure?	Permissions, row/column controls, aggregation thresholds, audit logging and restricted-data handling.
Evaluation approval	Is answer quality good enough for controlled user testing?	Golden questions, expected answers, failure categories, test results and remediation plan.
Pilot approval	Is the capability ready for controlled business use?	User support, caveats, known limitations, feedback process and acceptance criteria.
Production readiness approval	Is the service safe, supportable and economically viable?	Monitoring, support model, deployment process, cost controls, regression testing and named owners.
Scale approval	Should the capability expand to more users, domains or questions?	Usage, quality, incidents, cost, adoption, support load and unresolved risks.

The most important governance discipline is the willingness not to progress. A project should not move from POC to MVP simply because the natural-language interface works. It should progress only if the use case is valuable, the data is trusted enough, answer quality is evidenced, risks are controlled and the next stage has clear ownership.

5.2.1 Decision outcomes

At each decision gate, the question is not only whether the project can continue, but how it should continue. The standard outcomes should be:

Outcome	Meaning
Proceed	Evidence is strong enough to move to the next delivery step.
Narrow	Continue with reduced users, domains, questions, data scope or answer types.
Remediate / pause	Resolve material blockers before progressing.
Redirect / defer	Use another route, such as BI or analyst workflow, or move the item to a later backlog.
Stop	Do not continue this use case as a T2D candidate.

5.2.2 Illustrative evaluation expectations

Answer quality thresholds should be defined by use case risk, user group and decision impact. A low-risk internal pilot may tolerate more caveats and manual review than a production service used for senior reporting or regulated data.

As a practical guide, evaluation should distinguish between:

Category	Meaning
Correct answer	Answer matches the expected result and uses the approved definition.
Correct with caveat	Answer is directionally or numerically correct but requires a stated limitation, assumption or data caveat.
Clarification required	The system correctly identifies ambiguity and asks a follow-up question.
Unsupported	The system correctly refuses or redirects because the question is outside approved scope.
Unsafe request	The system refuses or restricts the answer because of access, privacy or exposure rules.
Wrong answer	The system returns an incorrect answer, wrong query, wrong definition or misleading explanation.

For production, teams should not rely on a single accuracy percentage. They should set minimum expectations for priority questions, high-risk question types and safe failure behaviour. For example, production should not proceed if material errors occur on priority questions, if unsafe requests are answered, or if wrong answers cannot be explained and remediated.

5.3 Budget, capacity and run cost

The final cost of a T2D initiative cannot be known precisely on day one because scope, data readiness, semantic complexity, access constraints and operating demand are still uncertain. A back-of-the-envelope estimate is still useful to support early prioritisation, funding discussions and proceed / pause decisions.

Note: The cost risk is not only the model bill. In T2D, cost can move into semantic preparation, warehouse execution,

repeated evaluation, trace retention, support load and rework caused by weak definitions.

5.3.1 Development and operational involvement

T2D budgets should cover more than the AI or model component. A realistic budget needs to include product ownership, business SME time, data and semantic work, engineering, architecture, security, evaluation, adoption and post-launch support.

The largest delivery effort is most often not prompt engineering or model usage. It is agreeing definitions, preparing trusted data, validating answers, controlling access, supporting users and maintaining the semantic layer.

The indicative ranges below assume a bounded enterprise use case, one primary domain, limited user groups, existing data platform capability and no major data remediation programme. Larger multi-domain, regulated or high-volume deployments will most likely require materially more effort.

Stage	Typical scope	Indicative delivery effort	Indicative Infrastructure / model cost
POC	Narrow use case, limited data, small test group, feasibility focus.	30 - 80 person-days	Low. Development/test environment, limited LLM/API usage, minimal monitoring.
MVP	Governed data layer, approved metrics, controlled user group, evaluation and security validation.	120 - 300 person-days	Medium. Environment costs, LLM/API usage, orchestration layer, logging, monitoring, semantic layer or BI tooling if applicable.
Production readiness	Production deployment, support model, cost controls, regression testing, access review and operational handover.	80 - 200 person-days	Medium to high. Production infrastructure, monitoring, alerting, audit logging, usage-based model costs and support tooling.
Ongoing run	Support, semantic maintenance, evaluation refresh, model/prompt versioning, cost and usage monitoring.	10 - 40 person-days per month	Usage-based. Driven by user volume, query complexity, token usage, data warehouse cost, monitoring and support needs.

These figures are planning ranges, not benchmarks or quotations. Their purpose is to help teams avoid underestimating the delivery and operating effort behind a production T2D capability.

5.3.2 Indicative cost per query

Per-query cost is difficult to estimate precisely before usage patterns are known. It depends on model choice, prompt length, metadata retrieved, result size, number of tool calls, data warehouse cost, caching, logging and whether the system needs clarification or retry loops.

As a planning assumption for a bounded enterprise T2D capability, teams can use the following indicative ranges:

Query type	Typical behaviour	Indicative cost per answered query
Simple governed query	One user question, limited metadata retrieval, one SQL query, short answer.	\$0.005-\$0.05
Standard analytical query	Metadata retrieval, SQL generation, validation, execution, result explanation and logging.	\$0.03-\$0.20
Complex / agentic query	Multiple reasoning steps, clarification, retries, larger context, multiple queries or richer explanation.	\$0.15-\$1.00+
High-volume optimised query	Cached metadata, smaller model, constrained prompts, efficient warehouse queries.	Below \$0.01-\$0.05

These ranges cover indicative variable usage cost only. They exclude fixed platform costs, internal support effort, security operations, maintenance and ongoing evaluation.

5.3.3 Key budget risks

- **Semantic effort is underestimated.** Metric definitions, joins, grains, filters and caveats take longer than expected.
- **Evaluation is treated as optional.** Without golden questions and expected answers, teams cannot evidence trust.
- **Security is added too late.** Access and exposure controls may require architecture rework.
- **Run cost is ignored.** Model/API usage, warehouse queries, logging, monitoring and support continue after launch.
- **Support ownership is unclear.** Without named owners, the service degrades as data, prompts, models and user behaviour change.

5.4 Typical team shape

The team should be shaped around the delivery stage. A POC can often be delivered by a small cross-functional team. MVP and production require broader involvement, especially around data ownership, semantic modelling, security, evaluation and support.

Role / capability	POC	MVP	Production readiness
Executive sponsor	Light involvement	Decision support	Gate approval / funding
Product owner	Core	Core	Core
Delivery lead / project manager	Light / part-time	Core	Core
Business SME / domain owner	Core	Core	Core
Metric owner	Part-time	Core	Core / governance
Data owner	Part-time	Core	Core / governance
Analytics engineer / semantic layer owner	Core	Core	Core
Data engineer	Part-time / core	Core	Core
AI architect / solution architect	Core	Core	Core
AI engineer / application engineer	Core	Core	Core
Security / governance lead	Light review	Core review	Core approval
QA / evaluation lead	Light	Core	Core
UX / user research	Optional	Part-time	Part-time
Change / adoption lead	Optional	Part-time	Core
Platform / DevOps / support owner	Light	Part-time	Core

A typical bounded MVP may therefore require a core team of **5-8 active contributors**, with additional part-time involvement from business, security, governance and platform stakeholders. Production readiness often requires fewer builders but stronger involvement from support, security, operations and ownership functions.

Note: *The exact team shape will vary, but the capability should not be staffed as an AI engineering task alone. T2D requires business, data, semantic, security, evaluation and operating ownership.*

5.5 Build, buy or partner considerations

The team and cost profile will depend on whether the organisation builds the capability internally, buys a platform, or uses a delivery partner.

Option	When it may fit	Main advantages	Main risks
Build internally	The organisation has strong data, AI, engineering and product capability.	Greater control over architecture, data flow, evaluation, security and roadmap.	Higher internal capacity demand; slower if skills or ownership are immature.
Buy platform / vendor product	The organisation wants faster time to value and can accept vendor patterns.	Faster start, clearer cost, packaged functionality, lower initial engineering burden.	Risk of lock-in, limited customisation, unclear evaluation depth, integration constraints.
Partner / contractor-led build	The organisation needs specialist delivery capacity or acceleration.	Access to expertise, faster delivery, knowledge transfer opportunity.	Risk of external dependency if ownership, documentation and handover are weak.
Hybrid approach	Common enterprise pattern: vendor or partner capability combined with internal ownership.	Balances speed with internal control.	Requires clear accountability across vendor, partner and internal teams.

For T2D, build-vs-buy should be assessed against specific delivery questions: who owns the semantic layer, how approved metrics are represented, how answer quality is evaluated, how access and exposure controls are enforced, how generated queries are validated, how failures are logged, how model or prompt changes are tested, and how easily the organisation can inspect or override the system's behaviour.

A vendor product may accelerate the interface, orchestration or model integration, but it does not remove the need for approved definitions, trusted data, access design, answer evaluation, adoption and support. These remain organisational responsibilities.

Buying a T2D interface does not outsource accountability for data trust, metric definitions, access rules, answer quality or operating support.

5.6 Operating model after launch

Production T2D needs an operating model, not just a deployment. After launch, the capability will change as users ask new questions, data sources evolve, metric definitions change, models are updated and failure modes are discovered.

The operating model should define ownership for six areas:

- **Product ownership:** scope, roadmap, prioritisation, user feedback and benefits tracking.
- **Data and semantic ownership:** metric definitions, joins, grains, caveats, source changes and semantic-layer updates.
- **Technical ownership:** orchestration, prompts, model versions, integrations, deployment and reliability.
- **Evaluation:** golden questions, regression tests, failure analysis and release checks.
- **Security ownership:** access reviews, audit logs, exposure controls, sensitive-data handling and policy compliance.
- **Support ownership:** incident handling, user support, monitoring, escalation and service communications.

The service should have a regular operating rhythm. This may include weekly issue triage during early adoption, monthly review of usage and failure patterns, periodic access reviews, regression testing before releases, and a controlled process for semantic or prompt changes.

The key operational risk is silent degradation. A T2D system can become less reliable when source data changes, definitions drift, user behaviour shifts, model behaviour changes or monitoring is ignored. The operating model exists to detect and correct these issues before trust is lost. Silent degradation should be monitored through explicit controls, including:

- **Regression tests** on golden questions before semantic, prompt, model or data-release changes.
- **Usage and failure monitoring** by domain, user group, question type and error category.
- **Data freshness and quality checks** on trusted sources and curated views.
- **Semantic change control** for metric definitions, joins, filters, grains and caveats.
- **Cost and latency thresholds** for query execution, model calls and retry loops.
- **Access and exposure reviews** to confirm that permissions and aggregation controls remain valid.
- **User feedback triage** to identify recurring confusion, wrong answers or missing definitions.

Metric definitions, semantic mappings, prompts and evaluation sets should be versioned. When a metric definition changes, the team should know which version was used for past answers, which tests need to be rerun, which users may be affected and whether historical answers remain valid. Material semantic changes should follow owner approval, regression testing and communication to affected users.

5.7 Adoption and trust

Adoption depends on trust. Users need to understand what the system can answer, what it cannot answer, how caveats are presented, and when analyst support is still required.

Analysts and data teams should be involved early. They help validate definitions, identify failure modes, interpret edge cases and maintain confidence in the capability. Without their involvement, T2D can be seen either as a threat to existing roles or as a shortcut around established data governance.

Training should focus less on “how to use a chatbot” and more on how to ask good questions, interpret answers, recognise caveats, provide feedback and escalate concerns. It should also address the probabilistic nature of LLM-based answers, so users neither over-trust nor under-trust the capability.

6 How to adapt the blueprint to a real project

This blueprint should not be applied mechanically. Each T2D initiative should adapt the delivery depth, governance, controls and artefacts to the intended use, business value, data maturity, risk profile and operating environment.

The key adaptation question is not “which steps can be skipped?” It is “what evidence is required before this capability is exposed to more users, more data or more decisions?”

6.1 Start with project intent

The first adaptation decision is the delivery intent. A proof of concept, MVP, pilot and production service require different levels of rigour.

Delivery intent	Main objective	Appropriate level of rigour
-----------------	----------------	-----------------------------

Discovery	Decide whether T2D is worth pursuing for a specific business problem.	Clear use case, target users, priority questions, early value case, initial risk and feasibility view.
POC	Test whether a narrow T2D flow is technically and commercially plausible.	Lightweight framing, limited source validation, narrow prototype, known caveats and explicit learning objectives.
MVP	Deliver a controlled first usable capability for a bounded user group and question set.	Governed data route, approved definitions, access model, evaluation approach, support route and known limitations.
Pilot	Test the capability with selected users in real workflows before broader release.	User feedback, usage evidence, answer-quality results, support process, incident handling and open-risk review.
Production readiness	Prepare the capability for live operation.	Monitoring, regression testing, security approval, cost controls, deployment process, support model and named owners.
Run / continuous improvement	Operate, support and improve the capability after launch.	Feedback loop, semantic updates, evaluation refresh, incident handling, roadmap governance and cost monitoring.

A common failure is treating a POC as if it proves production readiness. It does not. A POC may prove that the interaction pattern is possible; production requires evidence that it is reliable, secure, supportable and worth operating.

6.2 Calibrate delivery depth

Delivery depth should increase when the use case has higher risk, broader exposure or greater business impact.

Use stronger controls where the project involves:

- Sensitive or regulated data
- Large or diverse user groups
- Decision-critical metrics
- Complex joins, grains or metric definitions
- External or senior executive users
- High query volume or cost exposure
- Material dependency on third-party platforms or models

Use lighter controls where the project is exploratory, low-risk, internally contained and clearly labelled as non-production.

The discipline is to be proportionate without being casual. Lightweight delivery is acceptable for learning; it is not acceptable for uncontrolled production exposure.

6.3 Use decision gates, not fixed timelines

The blueprint should be adapted around evidence gates rather than fixed timelines. A phase is complete when the required decisions and evidence are in place, not when the planned date arrives.

For example:

- Framing is complete when the use case is valuable, bounded and owned.
- Data readiness is complete when source trust, metric definitions, joins, caveats and access routes are understood.
- Prototype is complete when the target flow works on approved questions and known limitations are documented.
- Evaluation is complete when answer quality has been tested against agreed questions and failure modes.
- Production readiness is complete when monitoring, support, cost control, security approval and ownership are in place.

If the evidence is weak, the project should narrow, pause, remediate or stop.

6.4 Decide what to simplify, strengthen or defer

When adapting the blueprint, teams should make explicit choices about which parts can be simplified, which need strengthening, and which should be deferred.

Adaptation choice	Appropriate when	Example
Simplify	Risk is low and learning speed matters.	Use a smaller question set for a short POC.

Strengthen	Risk, sensitivity or exposure is high.	Add formal security review, stricter access controls and more evaluation cases before pilot.
Defer	The item is not required for the current stage but must be addressed before scaling.	Defer multi-domain support until the first domain has proven value and trust.
Stop or redirect	The case for T2D is weak or foundations are not ready.	Replace T2D with a dashboard, semantic-layer project, data-quality remediation or analyst workflow improvement.

The output of adaptation should be a clear delivery stance: what the project will do now, what it will not do yet, what evidence is required to progress, and what risks are being accepted.

6.5 Practical adaptation checklist

Before applying the blueprint, teams should confirm the delivery stance:

- **Intent:** Is this discovery, POC, MVP, pilot or production?
- **Exposure:** Which users, data domains, decisions and risks are in scope?
- **Alternative route:** Why is T2D better than a dashboard, report, semantic-layer improvement or analyst-led process?
- **Evidence required:** What must be proven before the capability is exposed to more users, data or decisions?
- **Stop criteria:** What would cause the project to narrow, pause, redirect or stop?

The detailed approval criteria should then be managed through the decision gates in [section 5.2](#).

7 Final practitioner view

The hard part of T2D is not making a model produce an answer. The hard part is making sure all the answers are grounded in the right definition, the right data, the right permissions, the right context and the right level of confidence.

T2D should therefore be treated as a governed analytics product, not a chatbot feature. The delivery work sits around the model: framing the right use case, establishing data governance around trusted sources, definitions, ownership and access, building trusted semantic foundations, controlling exposure, evaluating answer quality, monitoring behaviour and maintaining ownership after launch.

The goal is not to make every question answerable. It is to make the right questions answerable, safely, reliably and with enough evidence that the organisation can trust the result.

-
1. Indicative timelines may be useful for planning, but they should not be treated as progression criteria. A phase is complete when the required evidence and decisions are in place. [↗](#)